Eberhard Karls
Universität
Tübingen

Seminar „Algorithmen für das WWW"

Vorstellung der Arbeit
„Recommendation systems: a probabilistic analysis"

von Robert Sösemann
am 12.12.2002

Ablauf der Präsentation

- Dauer: ca. 30 Folien, 45 min inkl. Fragen
- Schwerpunkt:
  - Motivation des Themengebiets *Collaboratives Filtern*
  - Vorstellung des Graphen-Modells
  - Vorstellung der Algorithmen

Motivation: Herausforderungen im E-Commerce

- Vorteile:
  - Kosten-/Zeitersparnis durch Automatisierung
  - Angebots-/Preistransparenz
  - Anonymität (gut für Kunden)
- Nachteile:
  - Anonymität (schlecht für Anbieter), weil...
    - falsche Lagerplanung
    - zielloses Marketing
    - Beratung fast unmöglich

Datentracking als Lösung?

- Aufbau riesiger Datenmengen durch
  - Zwangsregistrierung,
  - penibles Aufzeichnen von Kaufverhalten,
  - ständigem Feedbackzwang
- ist unerwünscht, da dies dem Wunsch nach Anonymität, Unbeeinflussbarkeit und Datenschutz widerspricht
- ZIEL: FINDEN VON ANONYMISIERTEN VERFAHREN FÜRS DATA MINING

Ist Beratung aber überhaupt möglich?

- Vergleich mit "Tante-Emma-Laden" hinkt
- Kundendatenbank + Verkäuferintuition = noch keine erfolgreiche Beratung, da
  - riesige potentielle Käuferschicht (weltweit)
  - Interessenvielfalt
  - theoret. unbegrenzte Produktzahl (eBay)
  - Korelationproblem bei vielen Produktarten (z.B. Musik und Reisen, Möbel und Essen)

Ausweg Collaboratives Filtern

- automatisierte Beantwortung der Frage: *"Welches Produkt interessiert Kunde X wahrscheinlich?"*
- Definition: Verfahren, um aus Datenbestand vieler Kunden (*kollaborativ*), dynamisch Produktähnlichkeiten zu extrahieren (*filtern*)
- Ziel: Finden von überdurchschnittlichen Häufungen (z.B. viele Kunden, die A kaufen, kaufen auch B)

Ausweg Collaboratives Filtern

- geht über bekannte Kategorisierungen (*Sci-Fi, Action, Hip-Hop, ...*) und Erfahrung hinaus
- funktioniert auch bei unterschied. Produktarten
- typische Anwendung Produktempfehlungen bei AMAZON
- zwei Arten des C.F.:
  - *aktives*: Datenbestand + aktives Feedback
  - *passives*: nur Datenbestand

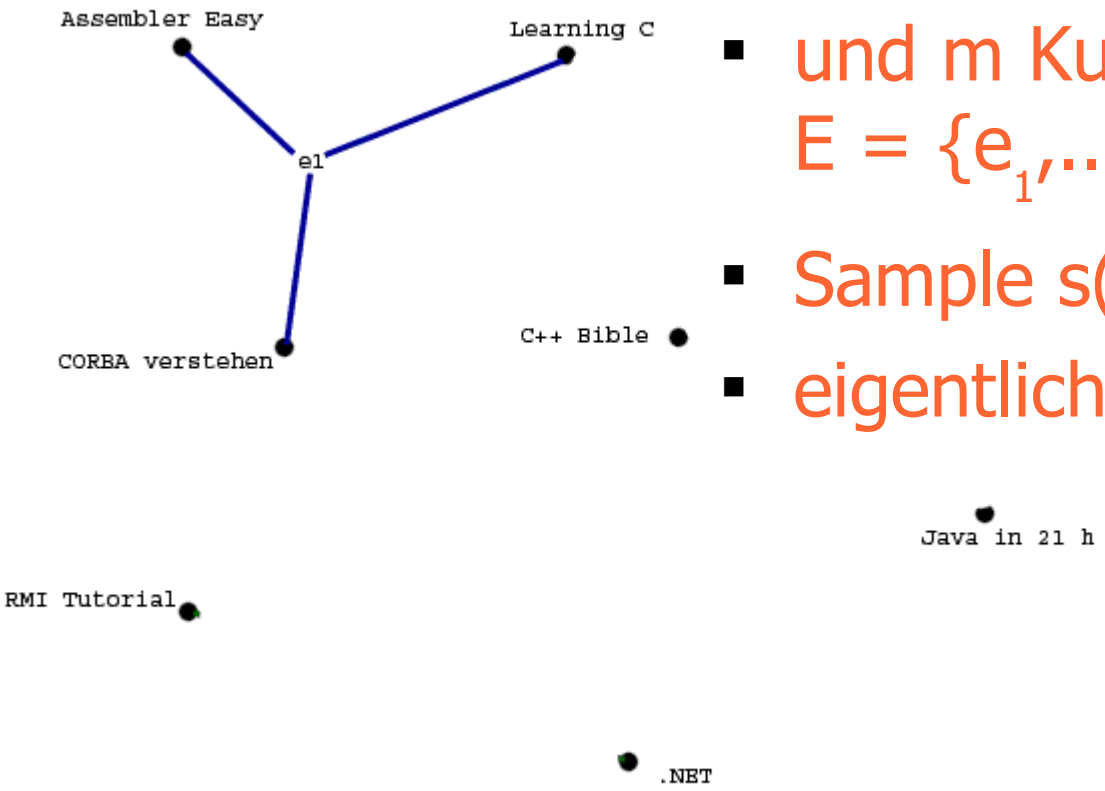
Ist das wirklich neu?

- auch Psychologie, Marketing u.a. beschäftigen sich mit Vorhersage von Kundenverhalten
- ABER: empirische Studien nicht algorithmierbar
- auch andere Gebiete des Data Mining (Information Retrieval, Expertensysteme, Clustering) extrahieren Muster aus Daten
- ABER: im Gegensatz zu C.F. immer feste Ähnlichkeitsmetrik (was heißt "nah")
- VORGESTELLTE ARBEIT SCHLIESST LÜCKE

Hypergraph-Modell und Notation

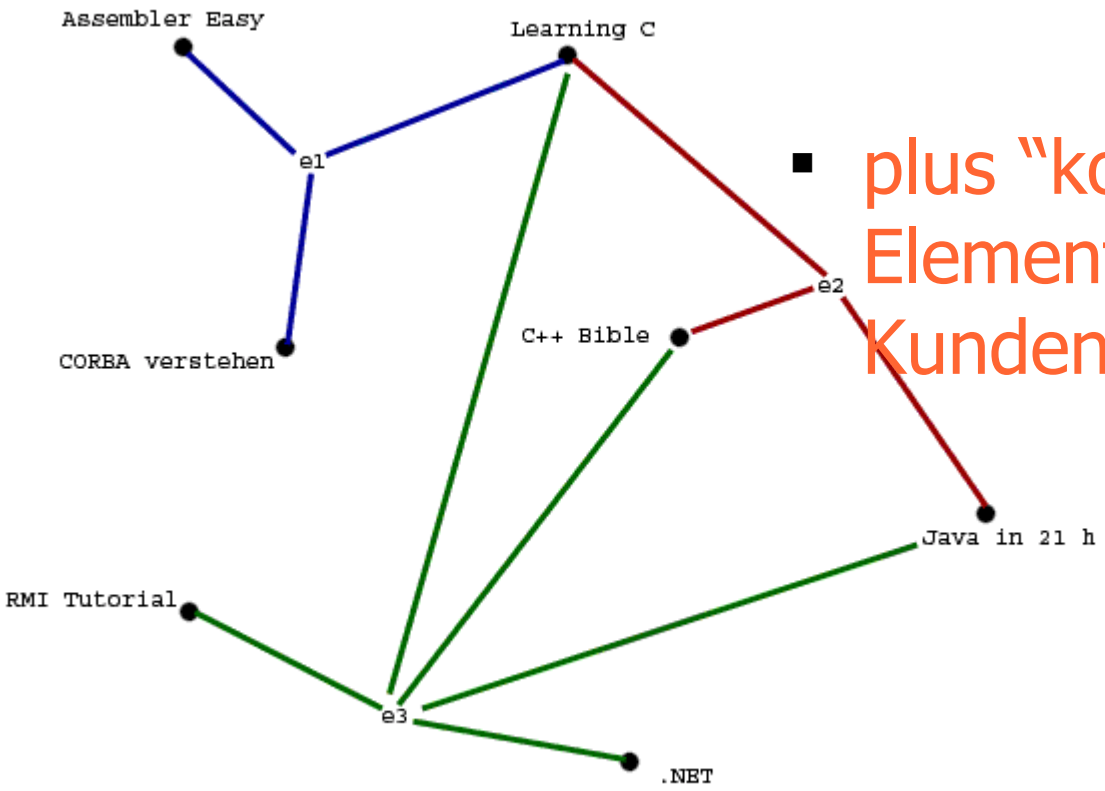
- Wir haben n Produkte
  Items $I = \{i_1,\ldots,i_n\}$

Hypergraph-Modell und Notation



- und m Kunden
  $E = \{e_1,\ldots,e_n\}$
- Sample s(e) als Kanten
- eigentlich Hyperkanten

Hypergraph-Modell und Notation



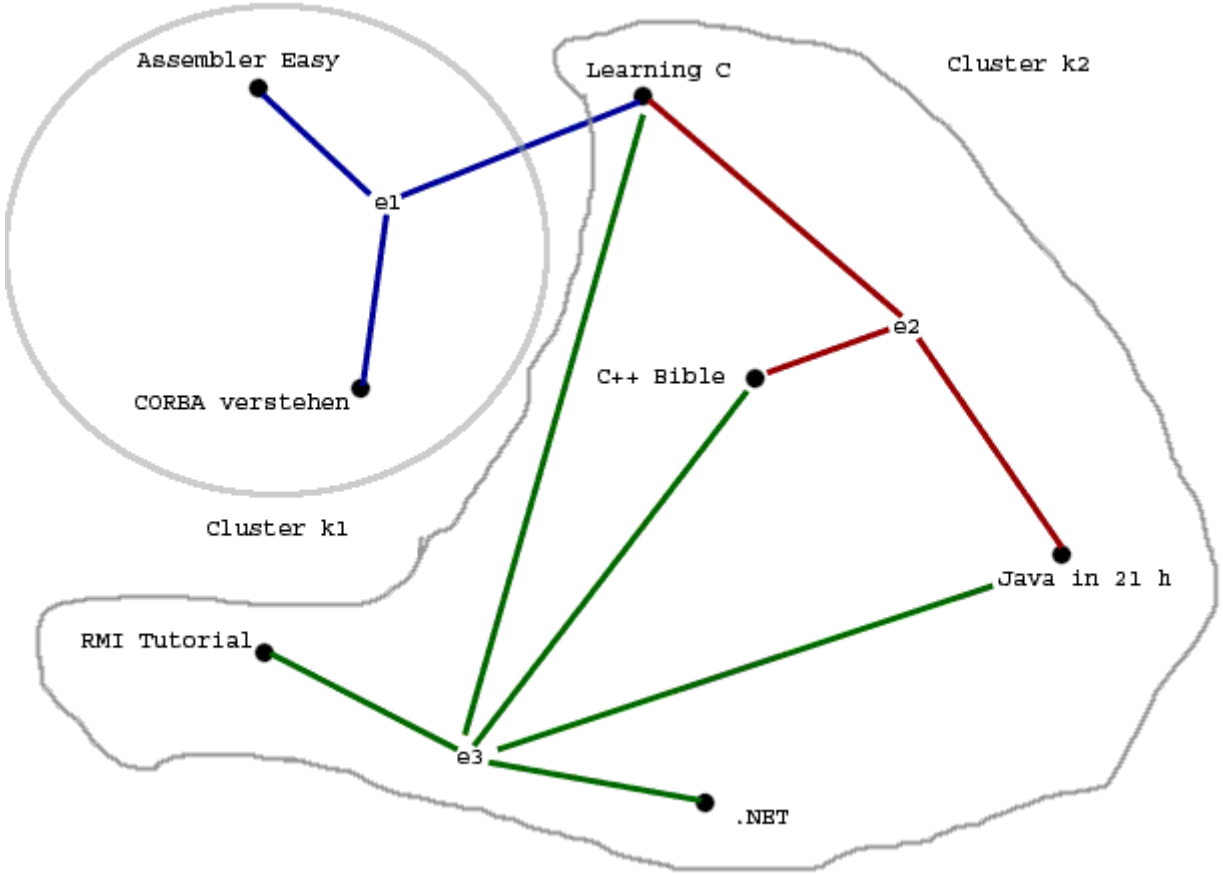
- plus “kollaboratives Element”, also weitere Kundensamples

Was ist im Modell Interesse?

- Kanten drücken Ähnlichkeit von Items aus
- d.h. durch “viele” Kanten verbunden Items bilden Interessenhäufung, sog. Cluster
- Vereinfachung im Modell: es werden keine Items, sondern nur Cluster betrachtet

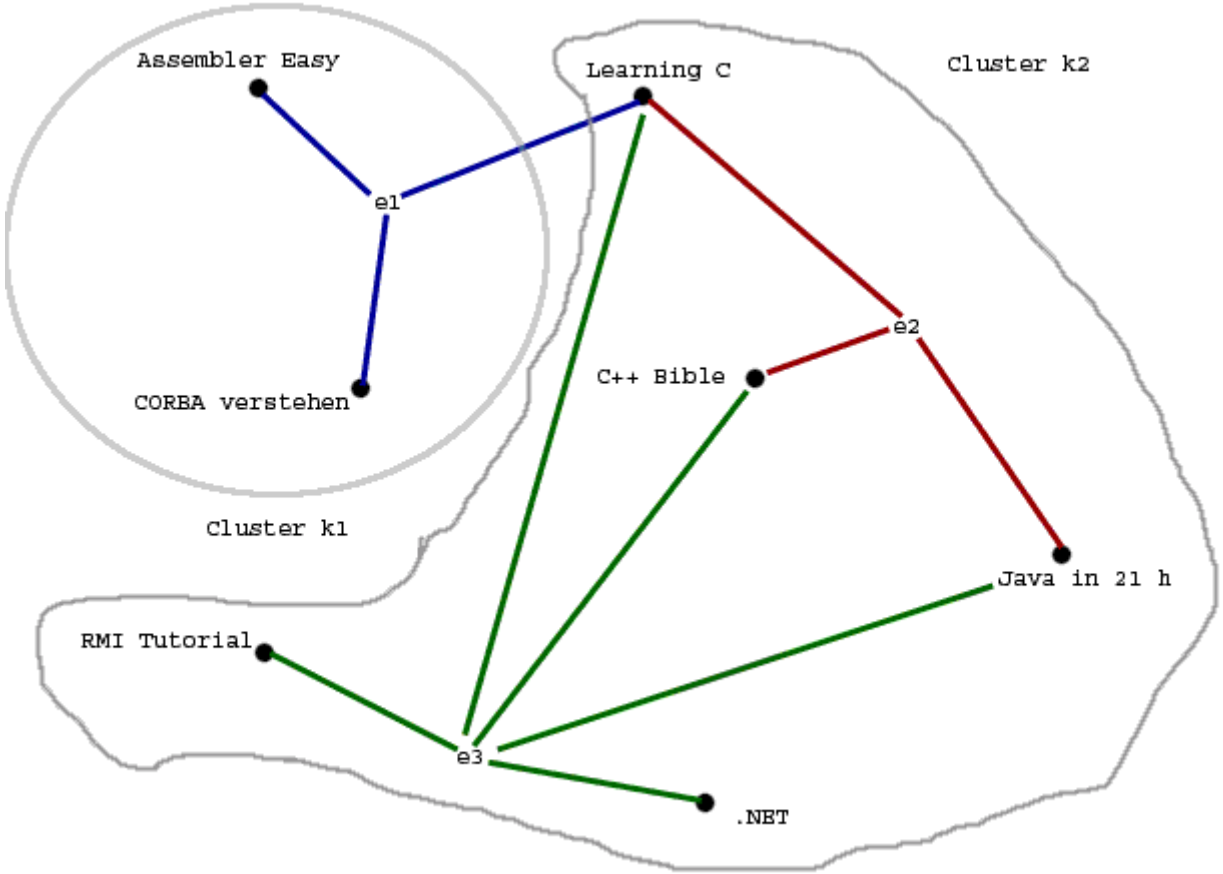
Modell und Notation

- Cluster bilden sich dynamisch aus Kundensamples
- Viele Kunden-kanten in Cluster = großes Interesse



Modell und Notation

- Clusterzuordnung in Funktion C : [item]->[cls#]



Wann ist Empfehlung nützlich?

- Bisher: nachträgliche Validierung
- Hier: Utilityfunktion direkt im Modell:
  U : [user] x [item] -> [0,1]
- Annahme: "echte Vorlieben" annähern durch bekannte Clusterwahl der Kunden anhand der Preferences-Vektor:
  - Kunde e_1: $p(e_1) = \{0.5, 0.5\}$
  - Kunde e_2: $p(e_2) = \{0.33, 0.66\}$
  - Kunde e_3: $p(e_3) = \{0.33, 0.66\}$

Wann ist Algorithmus gut?

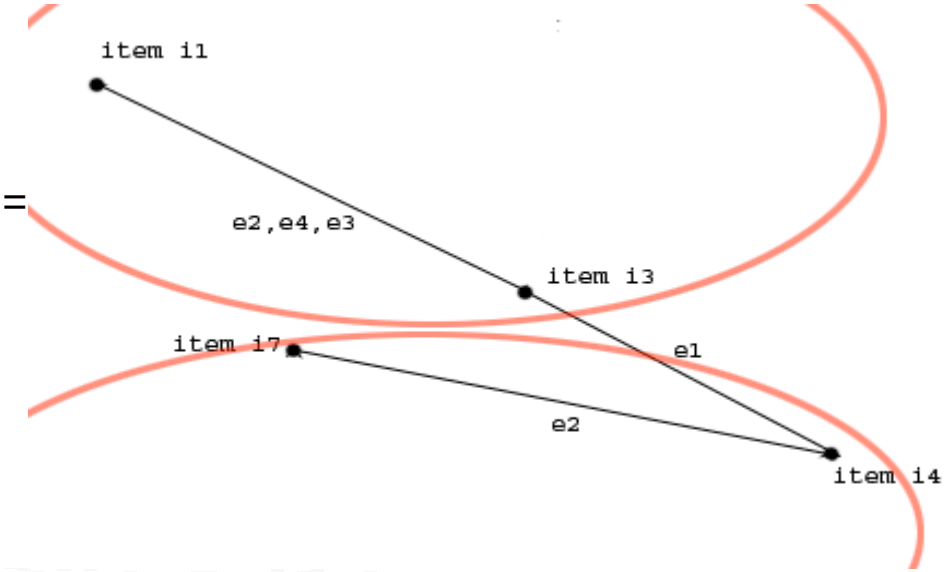
- ...maximiert die Gesamtnützlichkeit $\Pi(Alg,p)$
- ...d.h. maximiert $\Sigma_e\ p_k(e)$
- ...annähernd so gut wie optimale Benchmarks
- OPT: kennt Cluster und Preferences
  $\Pi(OPT,p)=\Sigma_e\ \max_{i=1..k}\ (p_k(e))$
- OPT_w: kennt nur Cluster
  $\Pi(OPT_w,p)$ abhängig von m, k,s,p
- bei m-> ∞ bzw. s-> ∞:OPT = OPT_w

Näherungsalgorithmen

- Einfach und gut (bzgl. Nützlichkeit)
- Algorithmen VIC, VOC, VRC
- Parameter: in e_i, out $cluster_k$
- haben noch Kenntnis von p und Clustern

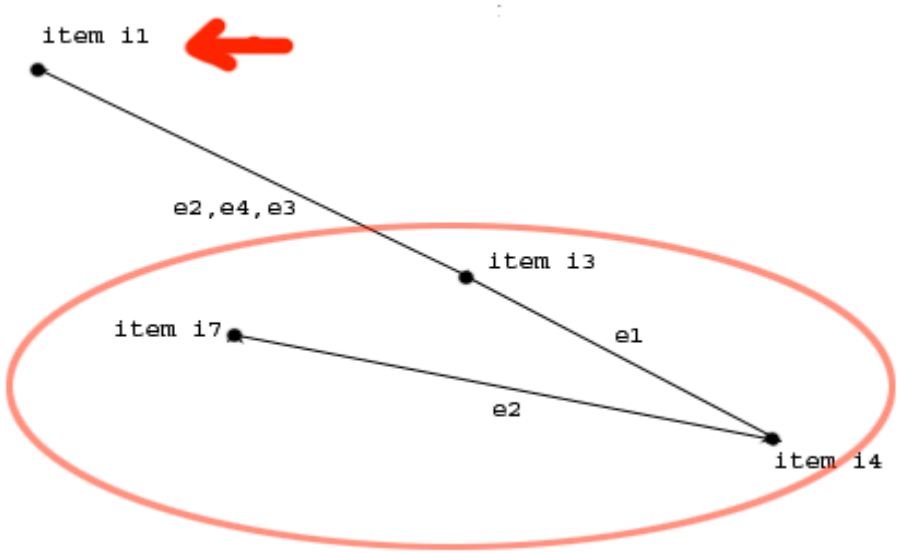
VIC – Vote in Cluster

```
cluster VIC(kunde) {

        sample1 = samplesArray[kunde][1];
        sample2 = samplesArray[kunde][2];

        cluster1 = C(sample1);
        cluster2 = C(sample2);

        //cross-edge
        if(cluster1 != cluster2)
                return preferedCluster ;
        else return cluster1;          //=
}
```



VIC – Vote in Cluster

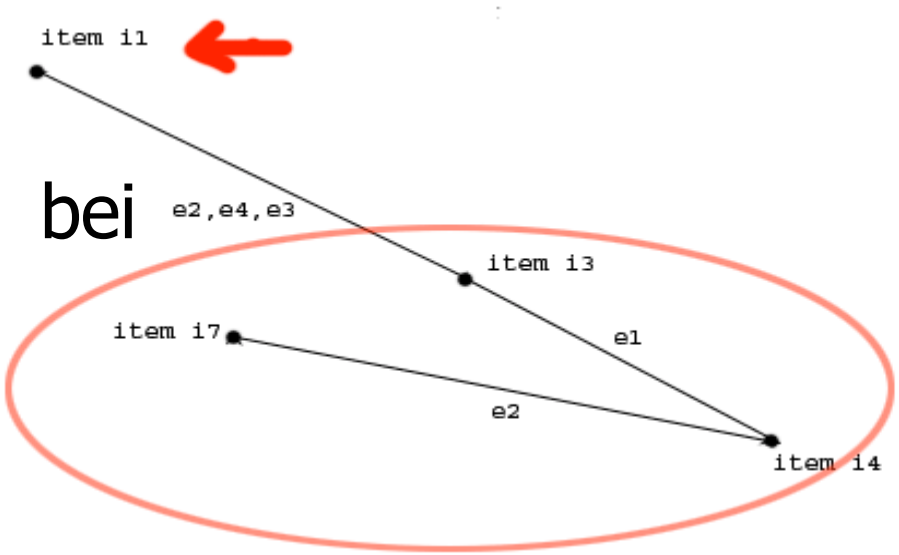
```
cluster VIC(kunde) {

        sample1 = samplesArray[kunde][1];
        sample2 = samplesArray[kunde][2];

        cluster1 = C(sample1);
        cluster2 = C(sample2);

        //cross-edge
        if(cluster1 != cluster2)
                return preferedCluster ;
        else return cluster1;          //=
}
```



VOC – Vote out of Cluster

```
cluster VOC() {
        return preferedCluster ;
}
```

- restriktiv gegenüber "Outsider-Kunden"
- sehr hohe Gesamtnützlichkeit bei "Massengeschmack"
- siehe Beispiel...



VOC – Vote out of Cluster

1.000.000.000 Kunden präferieren mit p={0.9, 0.1} Cluster k1
und
1000 Kunden mit p={0,1} Cluster k2.

Interessant: Behandlung der Kanten(Kunden) in Cluster 2:
Wird immer k1 empfohlen – also VOC - liegt die erwartet Nützlichkeit bei:

$$\Pi_{voc} = 0.1^2 * 0.9 * 1.000.000.000 = \underline{9.000.000}$$

9x höher als bei VIC mit:

$$\Pi_{vic} = 1000 + 0.1^2 * 0.1 * 1.000.000.000 = \underline{1.001.000}$$

VRC – Vote Randomly on Crossedge

```
cluster VRC(kunde) {
        //..wie VIC...

        if(cluster1 != cluster2)
                return randomCluster1Or2() ;
        else return cluster1;          //=cluster2
}
```

- Generalisierung VIC (Zufallswahl bei Crossedges)
- VOC zwar gut, aber Kenntnis des “besten” Clusters nötig
- Im Paper Beweis, dass: **Π (VRC,p) / Π (OPT,p) = 2/ sqrt(k) +1 und Π (VRC,p) = Π (VIC,p) für alle p**

MAX – mehr Samples ist besser

```
cluster MAX(kunde) {
        samplesInCluster[]; // wie viel Samples pro Kunde u. Cluster

        forall cluster
          forall sample {
                samplesInCluster[C(samplesArray[kunde][sample])]++;
          }

        sample1 = samplesArray[kunde][1];
        sample2 = samplesArray[kunde][2];
        cluster1 = C(sample1);
        cluster2 = C(sample2);

        if(samplesInCluster[cluster1]>samplesInCluster[cluster2])
                return cluster1;
        else if(samplesInCluster[cluster1]<samplesInCluster[cluster2])
                return cluster2;
        else
                return randomCluster1Or2();
}
```

MAX – mehr Samples ist besser

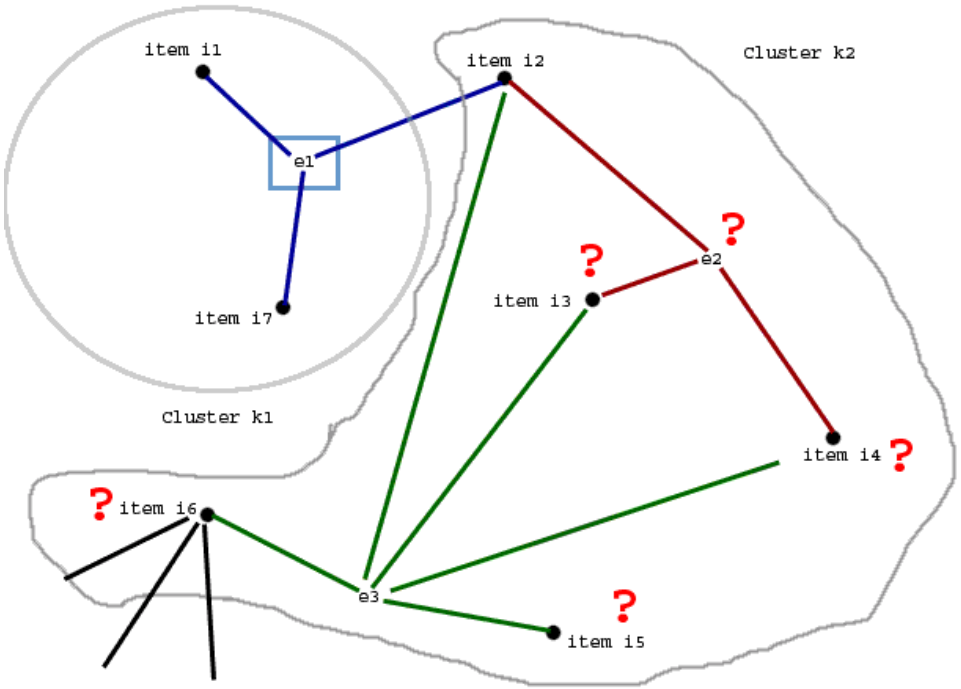
- Erweiterung von VRC bei s > 2, d.h. Kunde evtl. repräsentiert durch mehrere Kanten
- bei Wahl Cluster A oder B, wählen den, den Kunde bisher präferiert hat
- gut, da besser als Zufall und billiger als VRC
- Im Paper Beweis, dass Π(MAX,p) besser, je näher Kunde am “"Massengeschmack"” ist
- Im Paper Beweis, dass:
  Π (VRC,p) = (1-diff) * Π (OPT,p)

Algorithmen bei unbekanntem p u. k

- in Realität wissen wir viel weniger, da
  - k ändert sich dynamisch
  - p zu bestimmen sicher zu aufwendig
- trotzdem sind gute Empfehlungen machbar
- Algorithmen NEIGHBOR und VOTING
- Gesamtnützlichkeit:
  Π (NEIGHBOR,p) >= 0.7 * Π (OPT,p) und
  Π (VOTING,p) >= Π (NEIGHBOR,p)

NEIGHBOR

- Cluster nur implizit enthalten => Alg. empfehlen Items
- empfiehlt einem Kunden – repräsentiert durch eine ihrer Kante – zufällig eines der benachbarten Items

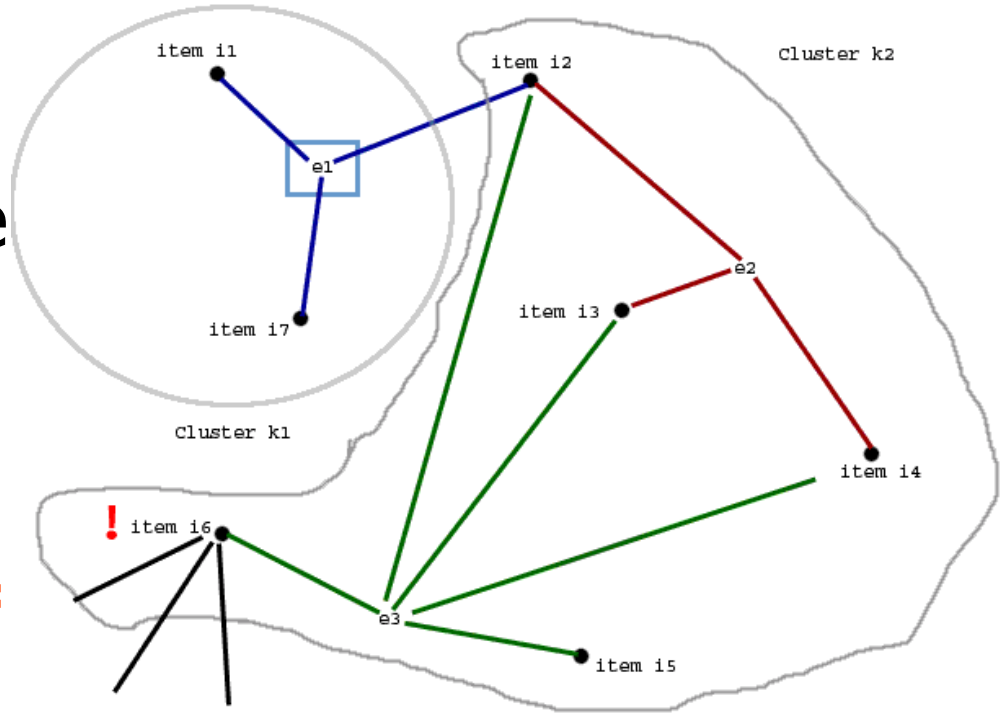


NEIGHBOR

```
item NEIGHBOR(kunde) {

 // Kanten des Graphs repräsentiert durch Nicht-0 in Matrix
 graph[kunde][knoten1][knoten2];

 //angrenzende Kanten bzw. Items finden
 forall andererKunde !=  kunde  {
          forall item != knoten1 {
                if(graph[andererKunde][knoten2][item] != 0)
                  return item;
                else if(graph[andererKunde][item][knoten2] != 0 )
                  return item
          }
          forall item != knoten2 {
                if(graph[andererKunde][knoten1][item] != 0)
                  return item;
                else if(graph[andererKunde][item][knoten1] != 0 )
                  return item;
   }
  }
}
```

VOTING

- Erweiterung von NEIGHBOR (kein Zufall)
- ...empfiehlt benachbarte Items, die durch größere Anzahl Kanten mit aktueller Kundenkante verbunden sind
- **$\lim_{m \to \infty} \Pi$ (VOTING,p) >= 2(sqrt(2)-1) * Π (OPT,p)**



VOTING

```
item VOTING(kunde) {

 // Kanten des Graphs repräsentiert durch Nicht-0 in Matrix
 graph[kunde][knoten1][knoten2];

 multiplicity[item];       // wie oft ist Item in Kante

 //angrenzende Kanten bzw. Items finden
 forall andererKunde != kunde {
           forall item != knoten1 {
                  if(graph[andererKunde][knoten2][item] != 0)
                    multiplicity[item]++;
                  else if(graph[andererKunde [item][knoten2] != 0 )
                    multiplicity[item]++;
           }
           forall item != knoten2 {
                  if(graph[andererKunde]knoten1][item] != 0)
                    multiplicity[item]++;
                  else if(graph[andererKunde [item][knoten1] != 0 )
                    multiplicity[item]++;
   }
 }
         multiplicity.sort();
         return multiplicity[first];         //den besten raus holen
}
```

Einschränkungen

- Modell eingeschränkt nutzbar, wg. unrealistischer Vereinfachungen
  - Cluster sind homogen – besser: Stufen, Hierarchien
  - Clusterwahl statt Itemwahl
  - Globales Optimierungsziel
  - Kunden sind „gleichwertig"
  - einseitige Utilityfunktion: besser auch Desinteresse darstellen durch negative Werte
- ABER: modulares Modell leicht erweiterbar
- Arbeit solider Grundstein für junges Gebiet!

Danke für die Aufmerksamkeit!
Noch Fragen?

Die schriftliche Ausarbeitung und diese Präsentation finden sich unter
www.webspace-journey.de/openproject